

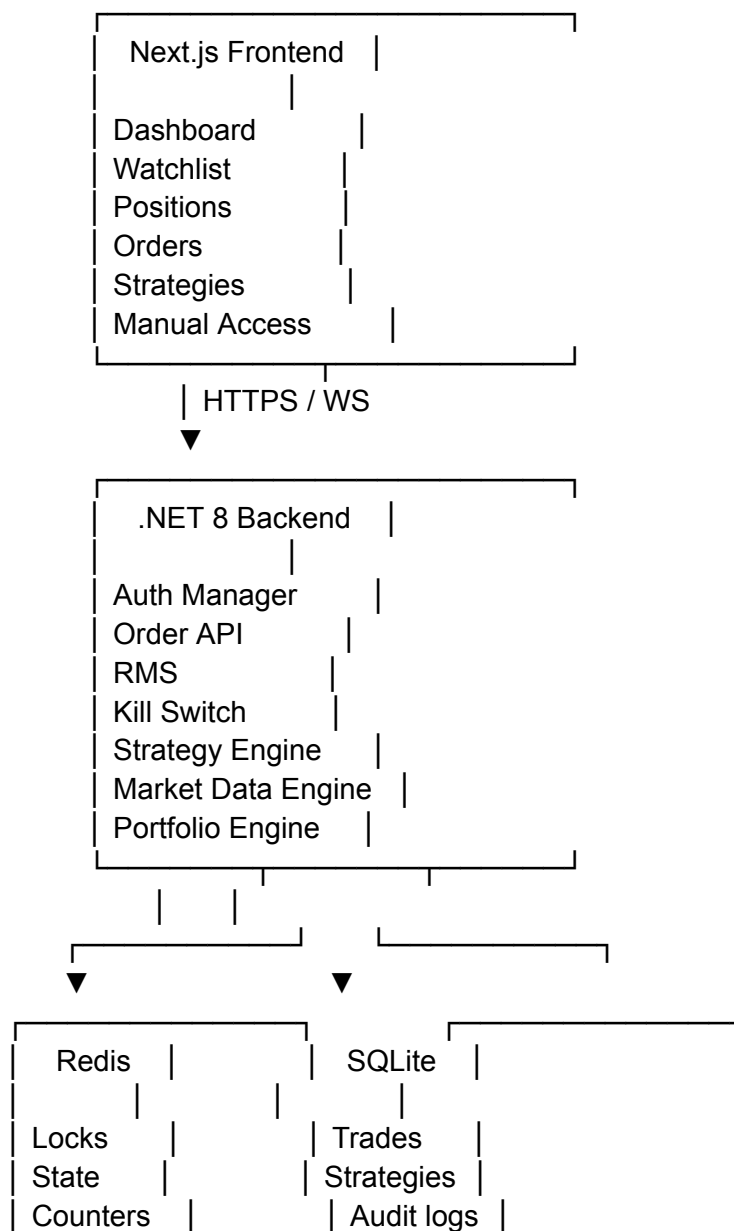
The biggest issue is that your current specification mixes **three different things**:

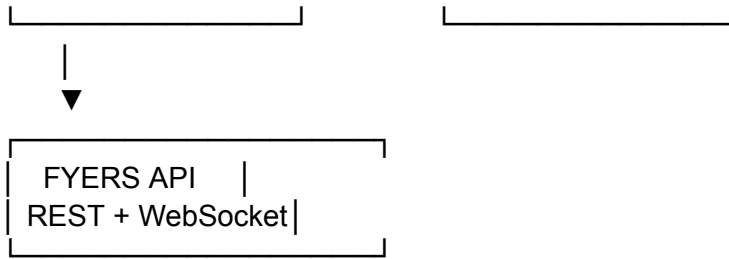
1. FYERS authentication/2FA
2. Your trading application's behavioral lock
3. The actual algo execution engine

They should be treated as separate security boundaries.

1. Your architecture is viable

A good production architecture would be:





And **nothing** in the browser should ever contain your FYERS secret, TOTP seed, or broker access token.

2. One major correction: don't design around "TOTP → access token"

This part of your specification needs to be corrected.

FYERS' documented User Apps flow is essentially:

Login → authentication → `auth_code` → token validation → `access_token`.

FYERS specifically documents the authorization-code flow for User Apps.

The TOTP itself is a **6-digit 30-second 2FA credential**, not the trading API access token.

More importantly, FYERS' current 2026 algo rules are significant:

- daily 2FA is required;
- continuous refresh-token sessions aren't supported under the new framework;
- orders must originate from a registered application mapped to a **whitelisted static IP**;
- API order rate is limited to **10 orders/second**;
- market orders are converted to MPP orders.

So your authentication module should be designed around **FYERS' currently supported authentication flow**, rather than assuming that your application can simply generate a TOTP and directly obtain a token indefinitely.

3. Your TOTP idea is actually useful — but use it as a behavioral gate

This is where I think your idea becomes particularly interesting.

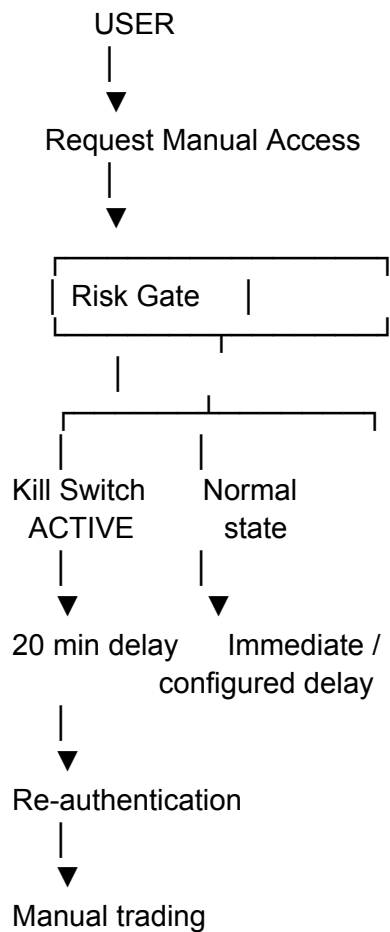
Instead of:

"Generate TOTP automatically so I don't need to interact with FYERS."

I would design it as:

"My trading application controls when I am allowed to authenticate/manual-trade."

For example:



That directly attacks the behavior you're trying to eliminate:

loss → emotion → immediate revenge trade

Instead:

loss → kill switch → forced cooling period → deliberate re-entry decision

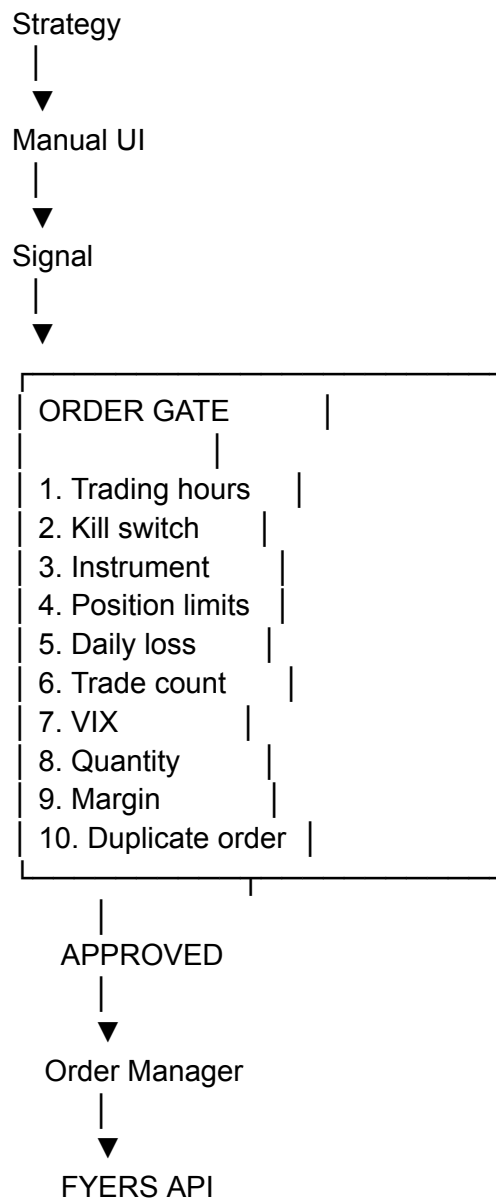
That's much more valuable than simply automating login.

4. I would change your RMS architecture

Don't implement RMS merely as ASP.NET middleware.

Make it a **central execution gate**.

Every order — regardless of origin — must pass through the same pipeline:



There should be no alternate route around this gate.

That includes your emergency/manual terminal.

5. I would add a second "Emergency Layer"

There is an important distinction between:

Master Kill Switch

Stop the algorithm.

and

Emergency Square-Off

Close everything immediately.

I would make them separate.

Kill Switch

ALGO_ENABLED = false
MANUAL_TRADING = false
NEW_ORDERS = false

Emergency Square-Off

cancel pending orders

↓

fetch positions

↓

calculate net quantities

↓

place closing orders

↓

verify execution

↓

verify positions = 0

↓

LOCK ACCOUNT

And the square-off process should be **idempotent**.

If your server crashes halfway through:

restart

↓

detect emergency_squareoff_pending

↓

reconcile FYERS positions

↓

continue square-off

Don't rely on an in-memory flag.

6. Redis should not be your source of truth

Your current design says Redis handles kill-switch state and SQLite handles history.

I'd go slightly further:

Redis

Use it for:

- current risk state
- distributed locks
- temporary authentication state
- trade counters
- cooldown timers
- real-time P&L cache
- WebSocket state
- idempotency keys

SQLite

Use it for:

- strategy configuration

- trades
- orders
- executions
- risk events
- kill-switch events
- authentication audit
- system events

But **FYERS remains the authoritative source for actual broker state.**

For example:

Your DB says:

NIFTY CE
+50 quantity

FYERS says:

0 quantity

Your system must reconcile to FYERS rather than blindly trusting SQLite/Redis.

7. Add a Reconciliation Engine

This is one of the most important things missing from your specification.

Have a background service:

BrokerReconciliationService

It periodically compares:

YOUR SYSTEM
vs
FYERS

for:

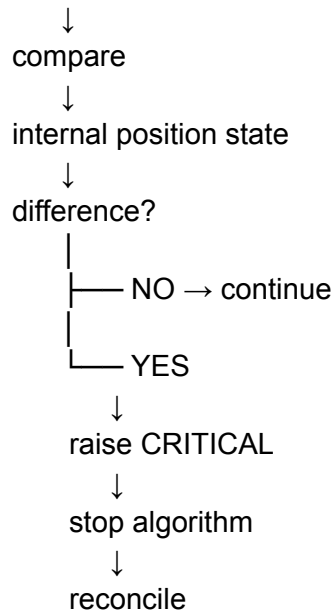
- positions
- orders
- executions
- available funds

- margin
- holdings

For example:

Every 5 seconds

FYERS positions



This protects you against crashes, network failures, duplicate requests and WebSocket disconnects.

8. Your strategy engine should NOT directly place orders

Don't do this:

```
if (supertrendBullish)
{
    fyers.PlaceOrder(...);
}
```

Instead:

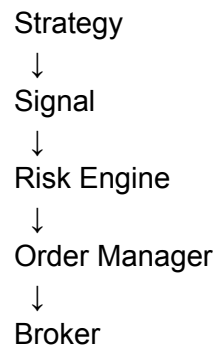
```
var signal = strategy.GenerateSignal(marketData);
```



```
if (signal != null)
{
    var decision = await riskEngine.Evaluate(signal);

    if (decision.Approved)
    {
        await orderManager.Execute(signal);
    }
}
```

This gives you:



This separation will become extremely valuable when you have 5–6 strategies.

9. Don't run all six strategies initially

This is another thing I'd change.

You currently have:

- ORB + VWAP
- EMA Pullback
- Bollinger Squeeze
- Supertrend
- NR7
- MACD

That's **too much for V1**.

I'd build the infrastructure first and initially activate only:

Strategy #1

Supertrend + VWAP

Then:

Strategy #2

ORB + VWAP

Then backtest/forward-test them.

Only after the execution infrastructure is proven should you add:

EMA

Bollinger

NR7

MACD

Otherwise you won't know whether performance comes from the strategy, execution, slippage, option selection or risk engine.

10. Your option-selection engine needs to be a separate module

This is particularly important because you're trading ITM NIFTY options.

Don't hardcode something like:

NIFTY + 2 strikes ITM

Instead create:

OptionSelector

with parameters:

Underlying = NIFTY

Minimum days to expiry

Maximum days to expiry

Target Delta

Minimum Delta

Maximum Delta
Minimum liquidity
Maximum spread
Minimum volume

Then:

NIFTY spot
↓
available expiries
↓
2–3 week expiry
↓
candidate strikes
↓
liquidity filter
↓
spread filter
↓
Delta filter
↓
selected contract

This is considerably more robust.

Also, don't assume the option will always have the desired delta. **The system should measure/filter it rather than infer it from "ITM."**

11. Your VIX rule should be configurable

Don't hardcode:

VIX < 11 → reject
VIX > 22 → reject

Store:

VIX_MIN = 11
VIX_MAX = 22

in the strategy/risk configuration.

Then you can test:

10–18
11–20
12–22
11–25

during backtesting.

And log the decision:

2026-08-17 10:15:32

Signal: BUY
Strategy: Supertrend
NIFTY: 24,xxx
VIX: 13.42

Risk:

Trading Window	PASS
Kill Switch	PASS
Daily Loss	PASS
Trade Limit	PASS
VIX	PASS
Instrument	PASS
Margin	PASS

DECISION: APPROVED

That audit trail will be extremely useful.

12. Your manual trading terminal should intentionally be "bad" at impulsive trading

This sounds counterintuitive, but it fits your objective.

Don't make it like a casino-style trading interface.

For example, when manually buying:

BUY NIFTY 24xxx CE

Quantity: [50]

Estimated cost: ₹xxxx

Risk per trade: ₹xxxx

Today's P&L: -₹xxxx

Trades today: 3 / 5

Daily loss remaining: ₹xxxx

[CONFIRM BUY]

And after a loss:

Last trade closed at: -₹4,850

Cooling period recommended.

Reason:

Daily loss utilization: 72%

[CONFIRM MANUAL TRADE]

Your software should **make disciplined behavior easier than emotional behavior**.

That's the real strength of this project.

13. One important infrastructure change

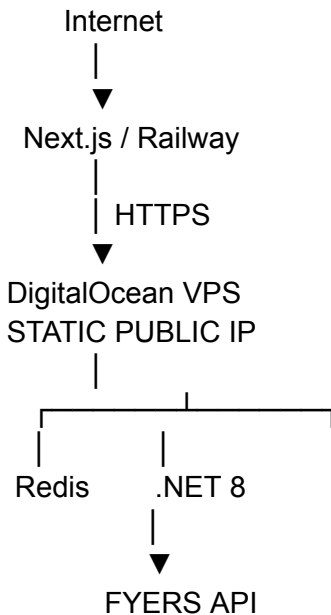
You specified:

Railway frontend + DigitalOcean backend

That's fine.

But because current FYERS rules require the trading API's orders to originate from a **registered app mapped to a whitelisted static IP**, the backend that actually sends orders should live behind the appropriate fixed/whitelisted public IP.

So your architecture should be:



Do not let Railway directly call FYERS order APIs.

Keep all broker execution on the VPS.

14. One more important 2026 consideration

FYERS states that under the current retail algo framework, third-party platforms must be empanelled and hosted within the broker's infrastructure.

For a **private application that you build for your own account**, this is a different situation from offering the platform to other traders. But before you turn this into a SaaS/product for other users, you should get the current FYERS/API compliance position confirmed directly with FYERS.

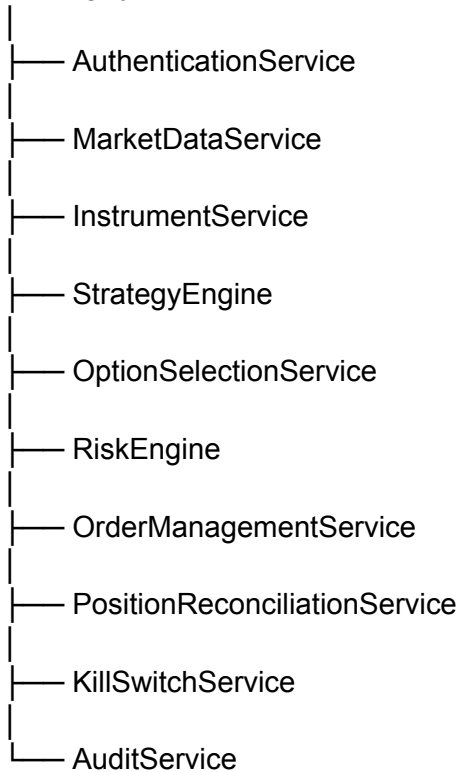
That distinction is important.

[FYERS API & Algo Trading guidance](#)

15. I would change your project into these 10 services

Instead of thinking "one trading bot", think:

TradingSystem



Then frontend:

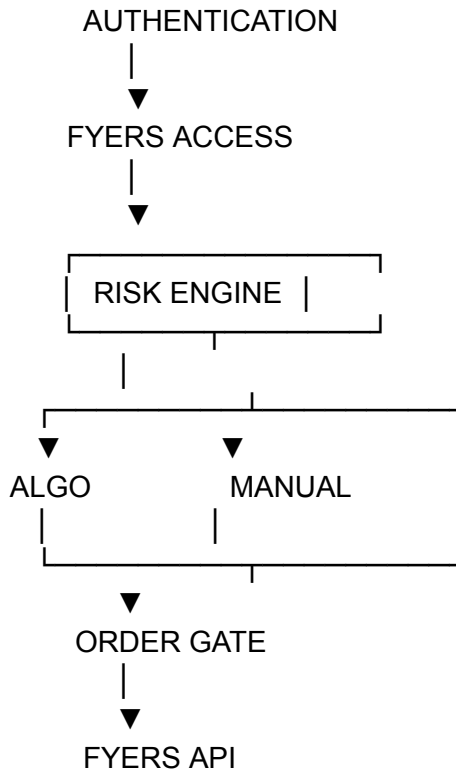
Next.js



Most important change to your original idea

I would **not** make the TOTP seed the mechanism that controls whether you can trade.

Instead:



And the **behavioral lock sits above both ALGO and MANUAL execution.**

That's the architecture I'd recommend for what you're actually trying to achieve.

Also, because FYERS' current API framework has changed substantially in 2026, I would **not start generating the final code from the old master prompt yet**. First we should update the specification against the current FYERS V3 authentication, static-IP, order-rate, WebSocket and daily-2FA requirements. FYERS itself points developers to its current V3 samples/documentation for the implementation details.